Student Task Manager

Second Demo Project Report

Safal Shrestha
S Q Abdullah Al Mamun

April 2026

Contents

1 Introduction **3**

2 Project Objective **3**

3 Scope of the Project **3**

4 Technologies Used **3**

5 System Design **4**
5.1 Frontend 4
5.2 Backend 4
5.3 Database 4

6 Core Features **4**

7 HTML Design and Wireframe Screenshots **5**
7.1 Login Page 5
7.2 Dashboard Overview 6
7.3 Task List and Filters 7
7.4 Add and Edit Task Form 8
7.5 Course Progress Section 9

8 Database Integration **9**
8.1 Database Schema 9

9 How the System Works **10**

10 Running Instructions **10**
10.1 Requirements 10
10.2 Steps to Run 11
10.3 Demo Login 11
10.4 Database Reset 11

11 Team Members and Contributions **11**

12 Challenges **11**

13 Future Improvements **12**

14 Conclusion **12**

1 Introduction

The Student Task Manager is a lightweight academic productivity web application designed to help students manage coursework, deadlines, and study tasks efficiently. The application allows students to create an account, sign in, and manage their own private task list. Each user can add tasks, update task details, track progress, and mark tasks as pending, in progress, or completed.

2 Project Objective

The main objective of this project is to build a simple but functional task management system for students. The system helps students:

- Create a private account and sign in.
- Add and organize academic tasks.
- Track task status as pending, in progress, or completed.
- Manage tasks using create, read, update, and delete operations.
- View progress using dashboard cards and a circular progress chart.
- Organize tasks by course, due date, and priority.

3 Scope of the Project

This version focuses on the core features needed for a strong second demo. It includes user authentication, private task storage, full CRUD task management, course organization, status tracking, search, filtering, and progress visualization.

Future versions could include calendar views, reminders, drag-and-drop task boards, file attachments, and advanced analytics.

4 Technologies Used

Category	**Technology**
Programming Languages	Python, HTML5, CSS3, JavaScript, SQL
Database	SQLite
Development Tool	Visual Studio Code
Browser Testing	Modern web browser

Table 1: Programming languages and tools used

5 System Design

The project uses a simple three-part architecture:

5.1 Frontend

The frontend is built with HTML, CSS, and JavaScript. It provides the user interface for login, registration, task creation, task editing, filtering, dashboard metrics, and progress visualization.

5.2 Backend

The backend is built using Python's built-in HTTP server. It serves frontend files and handles API routes for authentication, sessions, and task management.

5.3 Database

SQLite is used for persistent data storage. The database stores users, login sessions, and tasks. Each task is connected to a specific user through a `user_id`, which keeps task data private for each account.

6 Core Features

- User registration and login
- Private task list for each user
- Add new tasks
- View all tasks
- Edit and update tasks
- Delete tasks
- Mark tasks as pending, in progress, or completed
- Search tasks by title, course, or description
- Filter tasks by status and priority
- Dashboard summary cards
- Circular progress chart
- Course progress section

7 HTML Design and Wireframe Screenshots

The following screenshots show the HTML design of the Student Task Manager. These screenshots also serve as the wireframe for the second demo submission.

7.1 Login Page

login-page.png

Figure 1: Login and account access page

7.2 Dashboard Overview

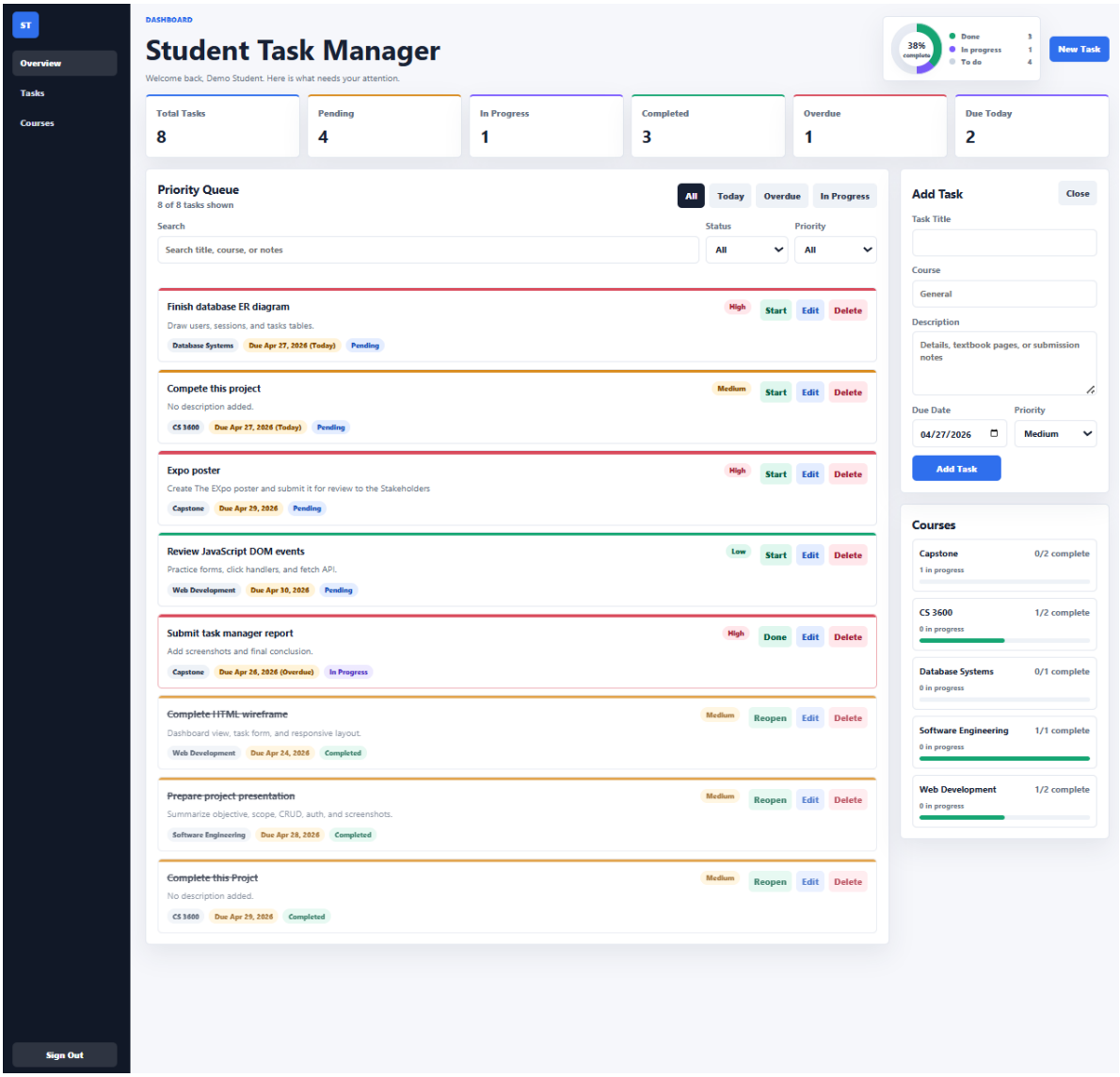

Figure 2: Dashboard with task summary cards and progress chart

7.3 Task List and Filters

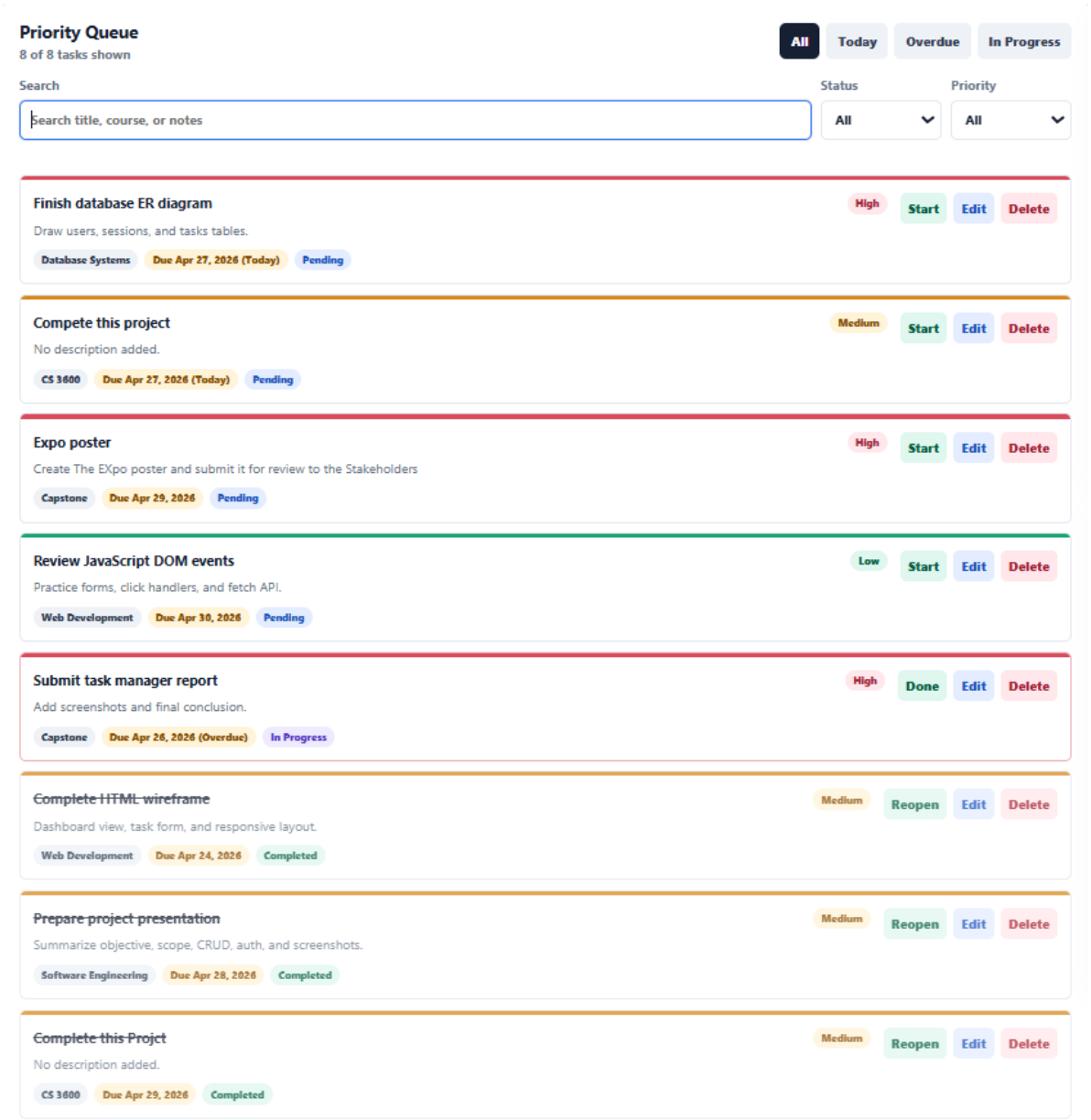

Figure 3: Priority queue with search, status filter, and priority filter

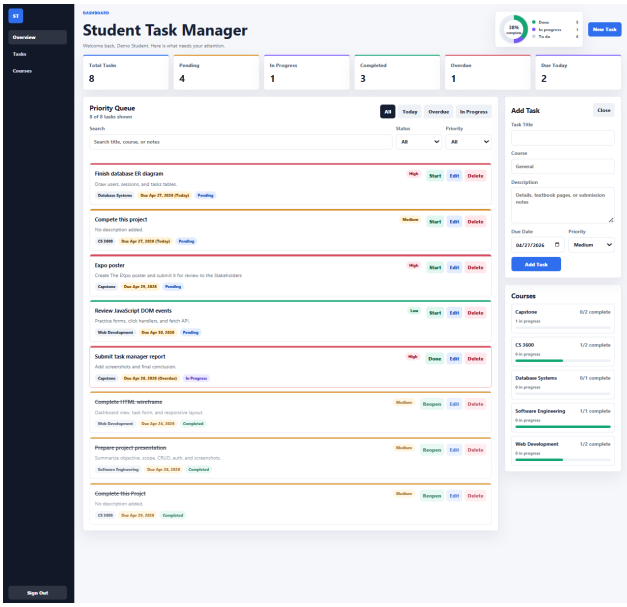

Figure 5: Enter Caption

7.4 Add and Edit Task Form

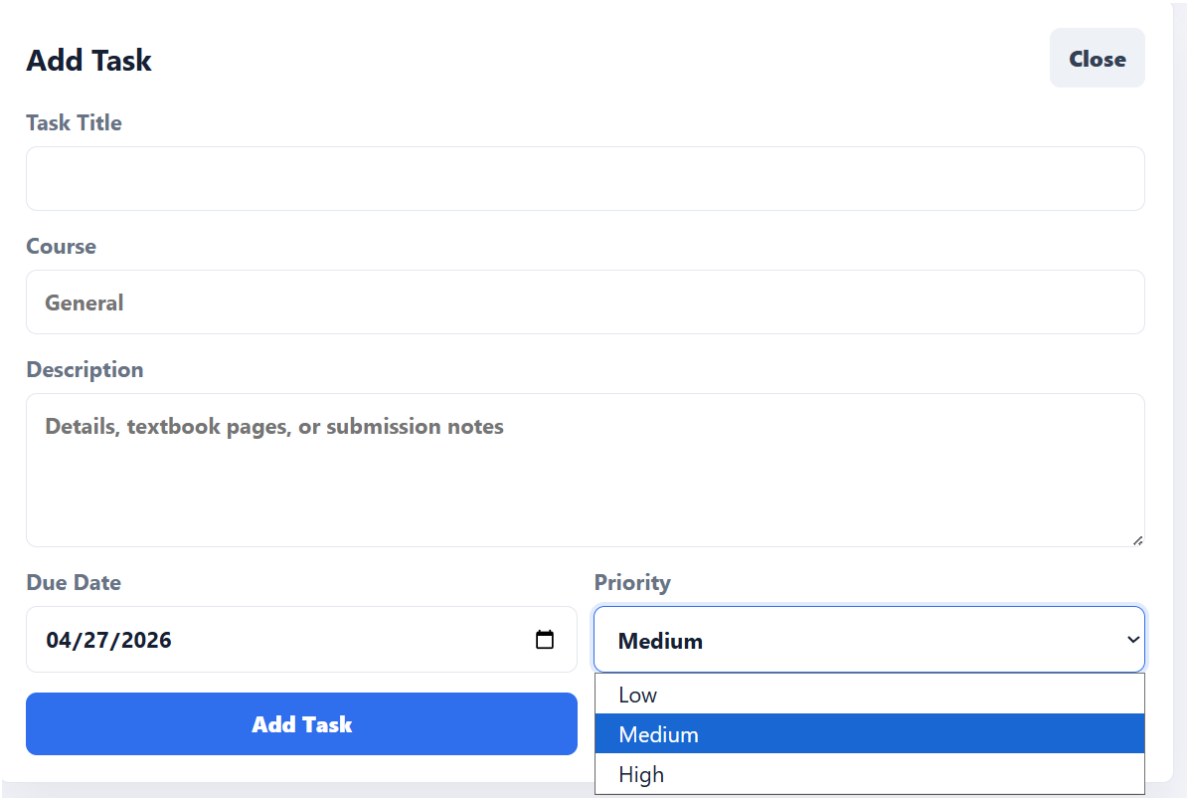

Figure 4: Task form for adding and editing student tasks

7.5 Course Progress Section

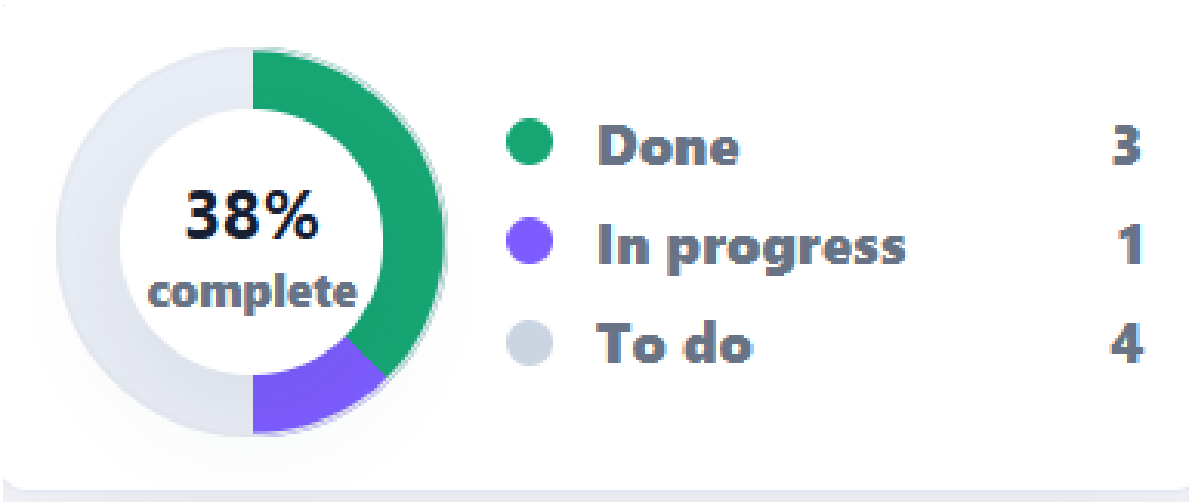

Figure 9: Course progress section showing completion by course

8 Database Integration

The project uses SQLite to store data locally. When the server starts, it creates the database tables if they do not already exist. The backend reads and writes task data through SQL queries.

The database contains three main tables:

- `users`: stores user account information.
- `sessions`: stores active login sessions.
- `tasks`: stores task records for each user.

8.1 Database Schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS sessions (
    id TEXT PRIMARY KEY,
```

```
    user_id INTEGER NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    course TEXT NOT NULL DEFAULT 'General',
    description TEXT NOT NULL DEFAULT '',
    due_date TEXT NOT NULL,
    priority TEXT NOT NULL CHECK (priority IN ('low', 'medium', '
       high')),
    status TEXT NOT NULL CHECK (status IN ('pending', 'inprogress',
       'done')),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

9 How the System Works

1. The user opens the web application in a browser.
2. The user signs in or creates a new account.
3. The frontend sends requests to the Python backend.
4. The backend checks the user session.
5. The backend stores or retrieves data from SQLite.
6. The frontend updates the dashboard dynamically.

10 Running Instructions

10.1 Requirements

- Python 3.12 or newer
- A modern web browser

10.2 Steps to Run

Open PowerShell in the project folder and run:

```
cd "C:\Users\shre4420\Documents\lab project demo\lab project demo"
python server.py
```

Then open the following address in a browser:

```
http://127.0.0.1:8001
```

10.3 Demo Login

```
Email: demo@student.local
Password: demo1234
```

10.4 Database Reset

To reset the database, stop the server, delete `mvp_task_manager.db`, and run `python server.py` again.

11 Team Members and Contributions

Team Member	Contribution
Safal Shrestha	Team leader, backend development, authentication, API routes, database integration, full-stack testing
S Q Abdullah Al Mamun	Frontend design support, UI layout feedback, documentation support, interface testing

Table 2: Team members and project contributions

12 Challenges

Some challenges during development included:

- Adding user authentication without using an external framework.
- Making sure each user only sees their own tasks.
- Connecting frontend actions to backend API routes.
- Updating dashboard counts and progress chart dynamically.
- Designing a clean interface that is simple enough for students to use.

13 Future Improvements

Future improvements could include:

- Calendar view for deadlines
- Reminder notifications
- Drag-and-drop task organization
- File attachments for assignments
- More detailed analytics and reports
- Team or group project task sharing

14 Conclusion

The Student Task Manager successfully demonstrates a practical academic productivity system. It includes user login, private task storage, full CRUD functionality, status tracking, course organization, and progress visualization. The project is lightweight, beginner-friendly, and built using Python, SQLite, HTML, CSS, and JavaScript.